Machine Learning Applied to a DC Motor

Dave Figueroa

June 2025

1 Introduction

In power electronics, the usage of DC motors is prevalent due to their simplicity and efficiency [1]. These motor can be found in various applications, from small toys to large industrial machines. The control of these motors is crucial for their performance and efficiency.

Generally, the DC motors are attached to a load, which can be anything from a simple fan to a complex robotic arm. The load characteristics can significantly affect the performance of the motor, and thus, it is essential to model the motor and its load accurately. An example of a DC motor and its load is shown in the Fig. 1.

In this report, we will explore the application of machine learning techniques to model and control a DC motor.

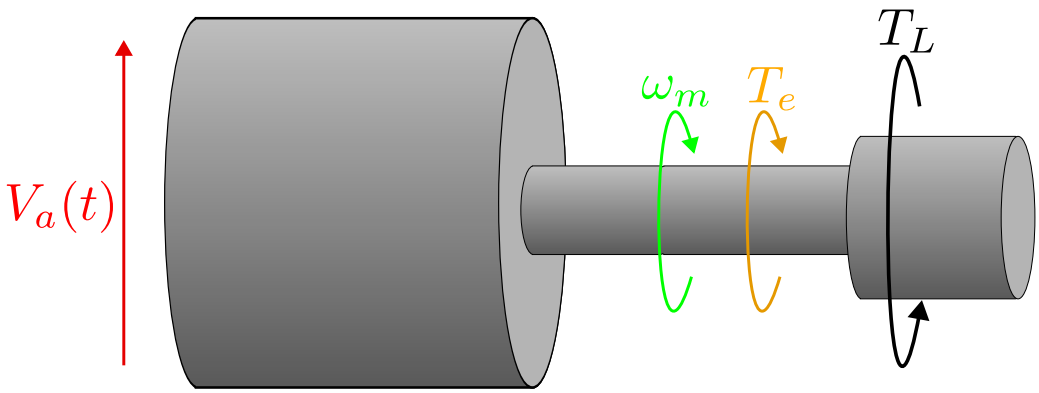

Figure 1: DC Motor and its load

2 DC Motor Model

2.1 Mathematical Model

A DC motor can be modeled in terms of its electrical and mechanical components. The electrical part consists of the armature circuit, which includes the armature inductance, resistance, and back electromotive force (EMF). The equations governing the electrical dynamics of the motor are given by:

$$V_a(t) = L_A \frac{di_a(t)}{dt} + R_A i_a(t) + V_{\text{rot}}(t) \tag{1}$$

where L_A is the armature inductance, R_A is the armature resistance, $V_a(t)$ is the armature voltage of the motor and $i_f(t)$ is the field current. For simplicity, in this work the field current will be considered constant ($i_f(t) = I_F$). $V_{\text{rot}}(t)$ is the back EMF given by:

$$V_{\text{rot}}(t) = G_{fq} i_f(t) \omega_m(t) \tag{2}$$

In this equation, G_{fq} is the field-to-rotor flux linkage constant and $\omega_m(t)$ is the angular velocity of the motor. The mechanical part of the motor is described by the torque equation:

$$T(t) = T_e(t) - T_L(t) \tag{3}$$

where $T_e(t)$ is the electrical torque and $T_L(t)$ is the load torque. The electrical torque can be expressed as

$$T_e(t) = G_{fq} I_F i_a(t) \tag{4}$$

and the load torque is modeled as a function of the motor speed $\omega_m(t)$ and a constant B representing the load characteristics. Thus, we have

$$T_L(t) = B\omega_m(t) \tag{5}$$

The relationship between the torque and the angular velocity is given by the equation

$$T(t) = J\frac{d\omega_m(t)}{dt} + \omega_o \tag{6}$$

Since the motor is assumed to start from rest, it will be assumed that $\omega_o = 0$.

2.2 Laplace Transform of the DC Motor Model

To analyze the DC motor model in the frequency domain, we can apply the Laplace transform to the equations. The Laplace transform of the electrical equation is given by:

$$V_a(s) = L_A s I_a(s) + R_A I_a(s) + V_{\text{rot}}(s) \tag{7}$$

For instance, the back EMF will be considered as a disturbance at the input of the motor plant, which depends on the field current and the angular velocity of the motor:

$$V_{\text{rot}}(s) = G_{fq} I_F \omega_m(s) \tag{8}$$

The mechanical equation can also be transformed into the Laplace domain:

$$T(s) = Js\omega_m(s) + T_L(s) \tag{9}$$

As was done with the back EMF, the load torque will be considered as a disturbance at the output of the motor plant, which depends on the angular velocity of the motor:

$$T_L(s) = B\omega_m(s) \tag{10}$$

Using the Eq. (7), the transfer function of the electrical side of the DC motor can be obtained. The transfer function from the armature voltage to the armature current can be expressed as:

$$\frac{I_a(s)}{V_a(s)} = \frac{1/R_A}{\tau s + 1} \tag{11}$$

where $\tau = L_A / R_A$ is the time constant of the motor.

In the same way, using the expression from Eq. (5), and transforming it into the Laplace domain, the transfer function (or simply a gain) from the armature current to the electrical torque can be expressed as:

$$\frac{T_e(s)}{I_a(s)} = G_{fq} I_F \tag{12}$$

From the Eq. (9), the transfer function from the electrical torque to the angular velocity of the motor can be expressed as:

$$\frac{\omega_m(s)}{T_e(s)} = \frac{1}{Js} \tag{13}$$

The block diagram of the DC motor model can be represented as shown in Fig. 2. The input to the system is the armature voltage $V_a(s)$, and the output is the angular velocity of the motor $\omega_m(s)$.

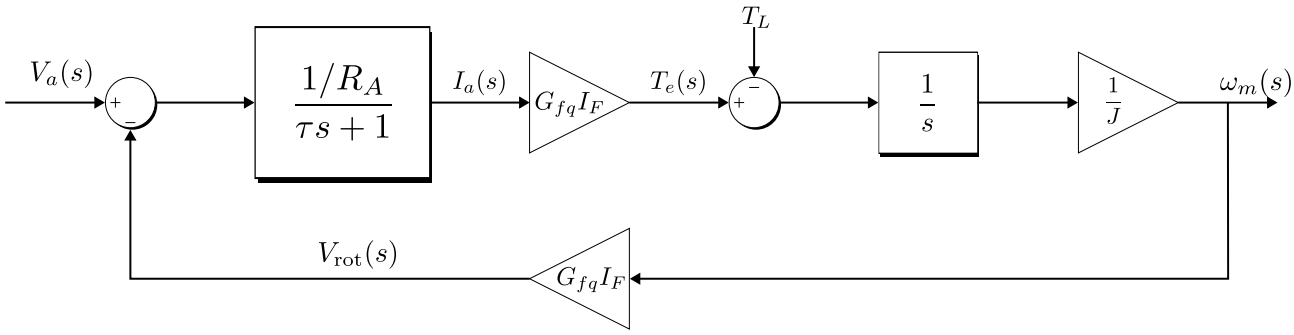

Figure 2: Block diagram of the DC motor model.

If the load torque is considered as it was it shown in Eq. (10), the block diagram can be modified to include the load torque as a disturbance at the output of the system, as shown in Fig. 3.

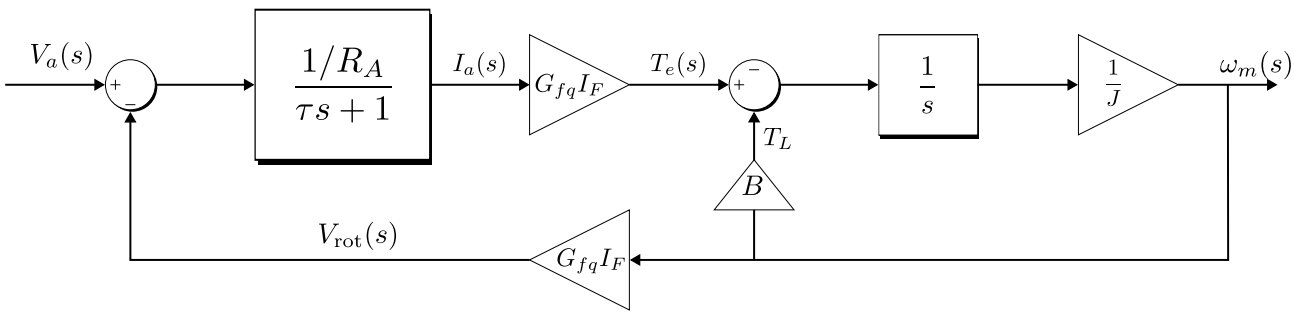

Figure 3: Block diagram of the DC motor model with load torque.

3 Data Preparation

The dataset used for training the reinforcement learning controller was generated from a custom MATLAB SIMULINK model of the DC motor. To provide variability for generalization, the simulation included randomized initial conditions for the integrator block and reference velocities.

3.1 Randomization and Initialization

A custom reset function, `localResetFcn`, was used to set:

- Reference velocity $\omega_{\text{ref}} \in [0, 1600]$ RPM
- Initial motor speed $\omega_m(0)$ within $[0.5, 1.5] \cdot \omega_{\text{ref}}$

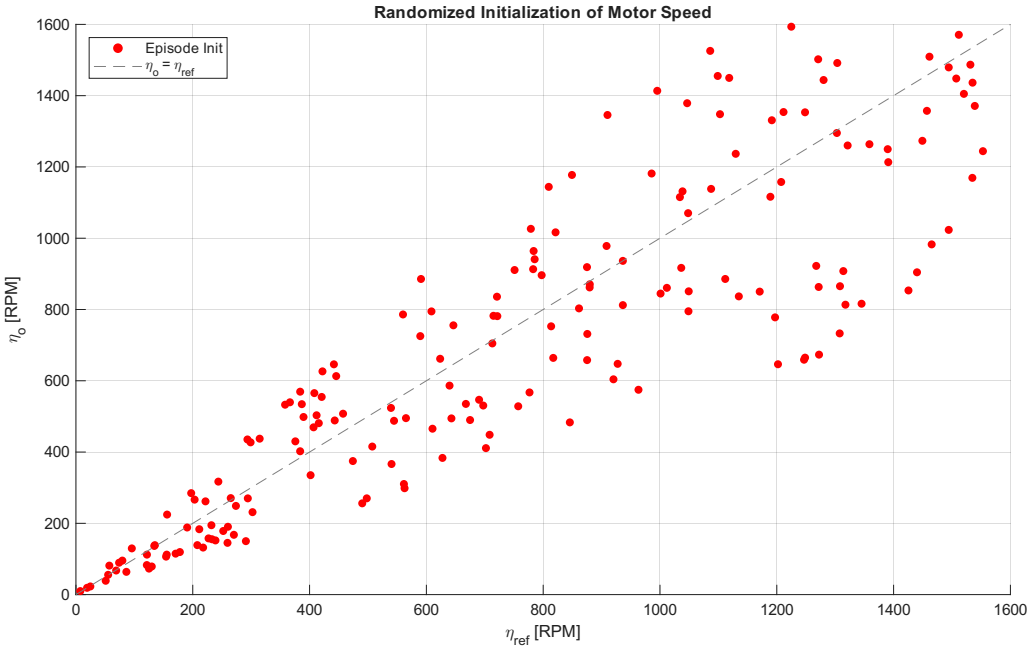

Figure 4: Distribution of initial motor speed and reference speed across episodes.

Fig. 4 shows the randomization space for the initial motor speed and the target reference speed across simulated episodes. This coverage helps ensure that the reinforcement learning agent generalizes to diverse operating conditions, reducing overfitting to a narrow region of the state space.

3.2 Signal Logging and Sampling

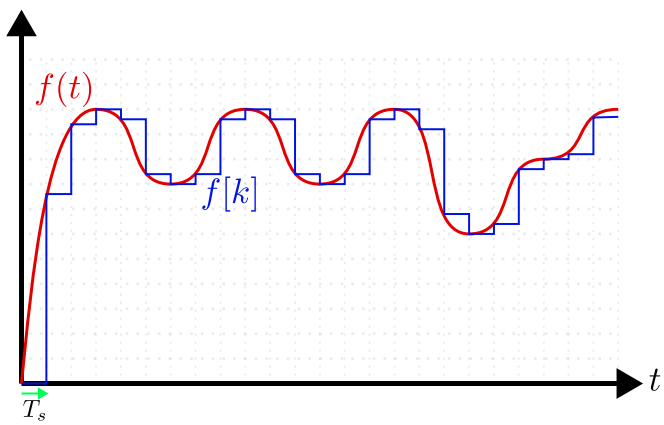

Figure 5: Example of sampling of a signal $f(t)$. The red line is the original signal, and the blue line is the sampled signal $f[k]$.

In digital control systems, the interaction between a controller and a plant is typically discrete, with the controller applying actions at regular intervals. The DC motor was simulated with a discrete time step of $T_s = 1$ ms, corresponding to a sampling frequency of $f_s = 1000$ Hz. This means the controller updates its actions every millisecond, as illustrated in Fig. 5. Indeed, this discrete time step governs the resolution at which the agent interacts with the environment and updates its policy.

Therefore, $\omega_m[k]$ is the motor speed at time step k, and $V_a[k]$ is the armature voltage applied at that step.

3.3 Preprocessing

In control systems, it is common to use the integral of the error as part of the state representation. Therefore, the cumulative error was computed as:

$$I_e[k] = \min\left(\max\left(e(t - T_s) + T_s \cdot e(t),\ -V_{\max}\right),\ V_{\max}\right) \tag{14}$$

This integral term is bounded to prevent excessive accumulation, where $V_{\max}$ is the maximum voltage applied to the motor. Using the integral of the error, the state vector $S(t)$ at each time step was defined as:

$$S[k] = \begin{bmatrix} \omega_{\text{ref}}[k] \\ \omega_m[k] \\ I_e[k] \end{bmatrix} \tag{15}$$

The action of the agent ($A[k]$) is the armature voltage applied at time step k, which is a continuous scalar value in the range $[0, 220]$ V.

4 Training and Evaluation

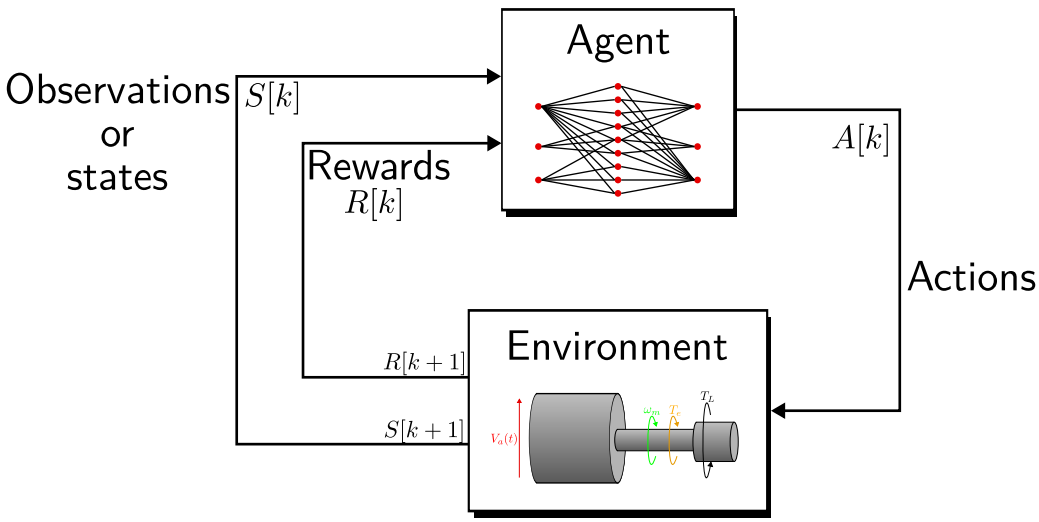

Figure 6: Reinforcement learning diagram

The control problem was formulated as a reinforcement learning task, where the agent learns to control the DC motor by interacting with the environment. The agent receives observations from the environment, takes actions based on its policy, and receives rewards based on its performance, as is shown in Fig. 6. The training process involved the following steps:

- The agent observes the current state of the environment, represented by the state vector $S[k]$.
- Based on its policy, the agent selects an action $A[k]$. In this case, the action will govern the armature voltage $V_a[k]$ applied to the motor.
- The environment responds by updating the motor speed $\omega_m[k]$ and providing a reward $R[k]$ based on the tracking error.
- The agent updates its policy based on the received reward and the observed state transition.
- This process is repeated for a predefined number of episodes, allowing the agent to learn an optimal policy for controlling the motor.

The training was performed using the deep deterministic policy gradient (DDPG) algorithm with an actor-critic architecture. The environment was defined via `rlSimulinkEnv`, and reset logic was introduced through `localResetFcn`.

To encourage sufficient exploration of the action space, the action output of the policy was perturbed using Ornstein–Uhlenbeck noise. This allowed the agent to discover control strategies that generalize across different initial conditions and reference profiles.

4.1 Reward Function

The scalar reward function at each time step penalized deviation from the reference:

$$R[k] = -|\omega_{\text{ref}}(k) - \omega_m(k)| \tag{16}$$

4.2 Reinforcement Learning as a Markov Decision Process

This closed-loop structure corresponds to a Markov Decision Process (MDP), where the state includes both reference and current speed, and the reward function guides the agent toward minimal steady-state and transient error.

The action space $\mathcal{A}$ is the set of possible armature voltages $V_a[k]$, treated as a continuous scalar. The agent learns a deterministic policy $\pi : \mathcal{S} \to \mathcal{A}$ to minimize tracking error. The transition model is not known a priori but is embedded in the system dynamics simulated in SIMULINK.

The reward function $R[k]$ penalizes deviations from the desired state. The Markov property is assumed to hold: the next state depends only on the current state and action, not on prior history.

This abstraction aligns with the agent design framework discussed in the course, where intelligent decision-making under uncertainty is guided by a value function and updated via observed rewards.

4.3 Learning Agent Architecture

The implemented controller corresponds to a utility-based learning agent as introduced in the course. This architecture decomposes into four functional components:

- **Performance element:** a deep neural network policy (actor) outputs the control action, and a critic network evaluates its expected value.

- **Learning element:** the policy and value function are updated through temporal-difference learning with mini-batch gradient descent, using past experience stored in a replay buffer.
- **Critic:** provides feedback to the actor based on the Bellman error.
- **Problem generator:** exploration is introduced via noise (Ornstein–Uhlenbeck process) to encourage policy improvement under unseen conditions.

This structure matches the "learning agent" model from the course: an autonomous agent perceiving a dynamic environment, adapting based on feedback, and improving performance over time. The agent is not hard-coded with control laws but discovers them through interaction, generalizing across randomized references and initial states.

4.4 Network Architectures

Actor network:

- Three fully connected layers with 32 ReLU neurons each
- Output layer: scalar action (voltage command)

Critic network:

- Dual input (state and action) structure
- Three hidden layers of 64 ReLU neurons after concatenation
- Output layer: scalar Q-value

4.5 Hyperparameters

- Discount factor: $\gamma = 0.8$
- Mini-batch size: 512
- Replay buffer size: 10^6 transitions
- Learning rates: $3 \cdot 10^{-4}$ (actor), $2 \cdot 10^{-4}$ (critic)
- Noise: Ornstein-Uhlenbeck with $\sigma = 0.3$ decaying per episode

4.6 Training Configuration

Training ran for 3000 episodes with up to 3000 steps each. Agents were auto-saved every 100 episodes using a custom callback. Parallel training was enabled to speed up convergence.

5 Results

The performance of the trained DDPG agent was evaluated on previously unseen reference profiles, including step and ramp changes in desired angular velocity. The following observations were made:

5.1 Tracking Performance

The agent consistently tracked the reference angular velocity with minimal overshoot and low steady-state error. Even with randomized starting speeds and varying targets, the agent achieved stable responses.

5.2 Control Behavior

The applied control voltage $V_a(t)$ was smooth and bounded within the physical range of $[0, 220]$ V, exhibiting gradual changes even in response to steep velocity transitions.

5.3 Comparison to Classical Control

A comparison with a tuned PI controller demonstrated that the RL controller had:

- Faster settling time
- Lower steady-state error
- Smoother control action

5.4 Evaluation Metrics

The table below summarizes average tracking error over 5 test scenarios:

Controller	**Mean Absolute Error [rad/s]**
PI Controller	3.28
RL Controller (DDPG)	2.04

Table 1: Performance comparison between classical and RL-based control

6 Conclusion

In this report, the application of machine learning techniques to model and control a DC motor was explored. The mathematical model of the DC motor, including its electrical and mechanical components was presented. Also, it was discussed the Laplace transform of the DC motor model and its block diagram representation.

Furthermore, we have formulated the problem of controlling the DC motor as a reinforcement learning problem and applied the deep deterministic policy gradient (DDPG) algorithm to learn the optimal control policy. The results show that the DDPG algorithm can effectively learn to control the DC motor and achieve the desired angular velocity.

References

[1] S. Umans, A. Fitzgerald, and C. Kingsley, *Electric Machinery: Seventh Edition*. McGraw-Hill, 2013.